

TP1 : un premier contact avec R et RStudio

Avant de débiter

Connectez-vous par navigateur au serveur RStudio du cours : <https://bif-rstudio.compbio.ulaval.ca>

Types de base en R

Les nombres

Les nombres sont représentés par la classe `numeric` dans R:

Dans les faits, il y a une distinction entre les `numeric`, qui incluent les nombres à virgule, et les `integer`, qui sont uniquement des nombres entiers. Généralement, la distinction n'est pas importante. Si on veut qu'un nombre soit considéré comme un entier, il faut ajouter un "L" à la fin.

On peut tester qu'une valeur est un entier avec la fonction `is.integer` et qu'une valeur est une valeur numérique avec la fonction `is.numeric`:

```
class(1L)
#> [1] "integer"
is.integer(1)
#> [1] FALSE
is.integer(1L)
#> [1] TRUE
is.numeric(1)
#> [1] TRUE
is.numeric(1L)
#> [1] TRUE
```

Les chaînes de caractères

Dans R, les chaînes de caractères peuvent être un seul caractère `ascii` ou bien plusieurs caractères. Il faut placer la chaîne de caractères entre double guillemets autrement R croit qu'on souhaite appeler une variable:

```
class("a")
#> [1] "character"
class("abc")
```

```
#> [1] "character"
class("1")
#> [1] "character"
class("")
#> [1] "character"
```

On peut combiner plusieurs chaînes de caractères avec la fonction `paste`:

```
a <- "salut"
b <- "bonjour"
paste(a, b)
#> [1] "salut bonjour"
```

Les booléens

Par définition, un booléen (logical) peut avoir seulement 2 valeurs: vrai (TRUE) ou faux (FALSE):

```
class(TRUE)
#> [1] "logical"
class(1 == 1)
#> [1] "logical"
class(1 == "1")
#> [1] "logical"
```

Les valeurs manquantes

Lorsque des données ne sont pas disponibles, on les représente par la valeur `NA`. R va également ajouter des `NA` automatiquement lorsque des champs sont vides lors du chargement d'un tableau.

On peut tester si une variable est non-disponible en utilisant la fonction `is.na()`.

```
is.na(NA)
#> [1] TRUE
is.na(1)
#> [1] FALSE
```

Structure de données: Vecteurs

Lorsqu'on souhaite représenter un ensemble de valeurs qui devraient être regroupées, on va utiliser des vecteurs. Les vecteurs sont donc une suite de valeurs du même type. La manière la plus simple de créer un vecteur est d'utiliser la fonction `c` (concaténation):

```
c(1,2,3)
#> [1] 1 2 3
class(c(1,2,3))
```

```
#> [1] "numeric"
c("a", "b", "c")
#> [1] "a" "b" "c"
```

Si on essayait de combiner des valeurs de différents types, R va les transformer dans le format le plus générique:

```
c(1, "a", 2)
#> [1] "1" "a" "2"
```

La seule exception est pour les valeurs manquantes, qu'on peut ajouter à un vecteur:

```
c(1, NA, 2, 3)
#> [1] 1 NA 2 3
```

À noter: les **NA** peuvent affecter les résultats de plusieurs fonctions telles que **mean**, **sum** ou **median**.

Il existe plusieurs fonctions qui permettent de créer rapidement des vecteurs de numériques sans avoir à écrire explicitement toutes les valeurs.

- **seq**: La fonction **seq** permet de générer tous les nombres entre 2 valeurs.

```
seq(1, 10)
#> [1] 1 2 3 4 5 6 7 8 9 10
seq(10, 1)
#> [1] 10 9 8 7 6 5 4 3 2 1
```

On peut également utiliser le paramètre **by** pour changer l'incrément:

```
seq(1, 10, by = 2)
#> [1] 1 3 5 7 9
seq(1, 10, by = 0.5)
#> [1] 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5 6.0 6.5 7.0 7.5 8.0
#> [16] 8.5 9.0 9.5 10.0
```

- **:**: Le **:** peut être utilisé pour obtenir un résultat similaire à la fonction **seq**:

```
1:10
#> [1] 1 2 3 4 5 6 7 8 9 10
10:1
#> [1] 10 9 8 7 6 5 4 3 2 1
-1:10
#> [1] -1 0 1 2 3 4 5 6 7 8 9 10
```

- **rep**: Pour répéter la même valeur plusieurs fois, on peut utiliser la fonction **rep**. À noter que cette fonction peut également être appelée avec les autres types de données:

```
rep(1, 10)
#> [1] 1 1 1 1 1 1 1 1 1 1
rep("a", 10)
#> [1] "a" "a" "a" "a" "a" "a" "a" "a" "a" "a"
rep(FALSE, 10)
#> [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

Opérations sur les vecteurs de valeurs numériques

Il est possible d'effectuer rapidement la même opération sur un vecteur de numérique comparativement à d'autres langages de programmation qui demanderaient l'utilisation de boucles.

```
vecteur <- 1:10
vecteur + 1
#> [1] 2 3 4 5 6 7 8 9 10 11
vecteur + rep(1, 10)
#> [1] 2 3 4 5 6 7 8 9 10 11
```

Dans le premier exemple, lorsqu'on écrit `vecteur + 1`, on dit en fait d'additionner la variable `vecteur` avec un vecteur de taille 1 (qui contient la valeur 1). Étant donné que le deuxième vecteur est plus petit que le premier, sa valeur est répétée pour obtenir la même taille.

Dans le cas où le deuxième vecteur serait d'une longueur plus grande que 1, l'ensemble du vecteur est répété:

```
vecteur + c(1,2)
#> [1] 2 4 4 6 6 8 8 10 10 12
vecteur + rep(c(1,2), 5)
#> [1] 2 4 4 6 6 8 8 10 10 12
```

Si le deuxième vecteur n'a pas une taille dont le premier vecteur est un multiple, la dernière répétition sera tronquée et R affichera un warning:

```
vecteur + c(1,2,3)
#> Warning in vecteur + c(1, 2, 3): la taille d'un objet plus long n'est pas
#> multiple de la taille d'un objet plus court
#> [1] 2 4 6 5 7 9 8 10 12 11
```

Autres opérations sur les vecteurs

Plusieurs fonctions sont disponibles pour calculer rapidement certaines statistiques sur les vecteurs. Il est recommandé de les utiliser plutôt que de calculer ces statistiques avec des boucles. D'une part, ça évite les erreurs et d'autre part, ces fonctions sont codées de manière à être extrêmement rapides.

```
sum(vecteur)
#> [1] 55
mean(vecteur)
```

```
#> [1] 5.5
median(vecteur)
#> [1] 5.5
length(vecteur)
#> [1] 10
```

Lorsqu'un vecteur contient des valeurs `NA`, le résultat de ces fonctions sera affecté:

```
sum(c(1,2,3,NA))
#> [1] NA
sum(c(1,2,3,NA), na.rm = TRUE)
#> [1] 6
```

Accéder à un ou plusieurs éléments d'un vecteur

On peut accéder aux éléments d'un vecteur en donnant l'index de l'élément souhaité dans le vecteur. On peut également sélectionner plusieurs éléments en même temps.

```
vecteur[1]
#> [1] 1
vecteur[2:4]
#> [1] 2 3 4
```

On peut également sélectionner les éléments qui correspondent à un certain critère:

```
i <- vecteur > 4
vecteur[i]
#> [1] 5 6 7 8 9 10
```

À votre tour

1. Assignez un vecteur `x` avec les valeurs numériques `(1,2,3,4,5)` et un vecteur `y` avec les valeurs `(4,4,4,4,4)`.
2. Essayez l'opération arithmétique `+` sur ces vecteurs
3. Assignez le résultat de l'opération à une 3e variable `z`
4. Regardez la liste des variables créées avec `ls()`
5. Regardez le contenu du deuxième élément du vecteur `z`
6. Quels sont les éléments plus petits que 7 dans le vecteur `z` (Essayez une comparaison logique).
7. Créez une matrice de deux colonnes et trois rangées avec les nombres `1:6` (PS : n'oubliez pas d'utiliser la documentation : `help(matrix)`).

Et si... ?

Pour chaque question, **prédisez d'abord la réponse sans exécuter**, puis vérifiez dans R. Si le résultat vous surprend, cherchez pourquoi.

Reprenez le vecteur `x` de la question 1.

1. Que renvoie `x[10]` ? Pourquoi R ne produit-il pas d'erreur ?
2. Que renvoie `x[-1]` ? Que fait un indice négatif ?
3. Et si `y` valait `c(1, 2)` au lieu de `(4,4,4,4,4)` : que renverrait `x + y`, et quel message R afficherait-il ?
4. Et si vous ajoutiez `byrow = TRUE` à la matrice de la question 7 : qu'est-ce qui change ?
5. Et si vous demandiez `matrix(1:6, ncol = 4)` ?

